

BEGINNER SOURCE GUIDE

DNF 自动化源码入门书

从“看不懂代码”到“知道程序到底怎么跑”的一本中文讲义

适读人群：完全不会写代码、但想理解这套 DNF 代码是如何工作的读者

讲解范围：窗口定位、截图识别、角色控制、打怪、拾取、门选择、小地图导航、A 星寻路、阈值与容差逻辑

基于源码目录：`D:\yolo\Yolov11\dnf`

这份说明书不是教你“怎么写出同样的程序”，而是帮你把现有源码拆成一个个容易理解的小部件。你只要记住一句话：这套程序的核心链路是“找到游戏窗口 -> 截图 -> 识别画面 -> 在小地图上决定下一间房 -> 生成一个方向 -> 按键控制角色去战斗、捡东西、过门”。

目录

1. 先用一句人话说清这套程序
2. 程序总架构：各个文件分别负责什么
3. 程序是如何绑定窗口的
4. 程序是如何检测游戏画面的
5. 程序如何打怪、拾取物品、控制角色移动
6. 程序是如何寻路的
7. A 星算法是如何和整个程序关联起来的
8. 所谓“容差算法”在这套代码里到底是什么
9. 重要算法逐个解释
10. 这套源码的优点、局限和你看代码时最该抓住的重点

1. 先用一句人话说清这套程序

这套程序不是“直接读游戏内存后自动通关”的风格，而是更接近“看屏幕做判断”的风格。它每一帧都做同样几件事：

1. 找到 DNF 窗口并截图。
2. 用 YOLO 模型识别角色、怪物、门、金币、掉落物。
3. 另外单独截取小地图区域，用模板匹配找到“当前房间、Boss 房、问号房、精英房、下层入口”。
4. 把“小地图上的当前位置”和“目标房间”丢给 A 星算法。
5. A 星不输出一整条像素路线，只输出“下一步应该往上/下/左/右哪个房间走”。
6. 控制层根据这个方向，在当前画面里找对应方向的门，然后按方向键移动。
7. 如果屏幕里还有怪物，就优先去打怪；如果有掉落物，就优先去捡；都没有才去找门进下一间房。

一句话总结：这不是“纯寻路程序”，而是“检测 + 决策 + 按键执行”的组合系统。

2. 程序总架构：各个文件分别负责什么

文件	职责	你可以把它理解成什么
main.py	主循环，负责截图、调用检测器、调用导航器、调用动作执行器	总导演
detector.py	YOLO 目标检测	眼睛 1 号，负责看大画面
minimap_nav.py	小地图裁剪、模板匹配、目标房间选择、A 星调用	眼睛 2 号 + 导航员
minimap_astar.py	A 星搜索和下一步方向判断	路线规划器
door_strategy.py	从多个门里挑一个最合适的门	门的筛选器
game.py	战斗、拾取、过门、卡住恢复、方向按键控制	手和脚
olaplug	一个 DLL 封装，包含更底层的窗口绑定和输入接口	备用工具箱，但当前主流程没有用上

最重要的一条调用链

```
main.py
├─ _find_dnf_window()      找窗口
├─ _capture_client()      对游戏客户区截图
├─ detector.detect()      识别玩家 / 怪 / 门 / 物品
├─ navigator.build_route_snapshot()
│   └─ detect_room_markers() 看小地图
│       └─ next_direction()  调用 A 星
│           └─ choose_best_door() 估分选门
└─ Game(...).run()      战斗 / 捡东西 / 进门 / 恢复卡住
```

非常关键：代码里虽然有 `OLAPlugServer.BindWindow()` 这种真正意义上的“后台绑定窗口”接口，但当前主流程没有调用它。现在实际运行依赖的是 `win32gui`、`pyautogui`、`pydirectinput`。

3. 程序是如何绑定窗口的

这里要先纠正一个很容易误解的点。你问“程序如何绑定窗口”，如果按字面理解，很多人会想到“注入、后台绑定、DLL 绑句柄”。但这份源码当前真正做的，是比较轻量的“逻辑绑定”：**先找到 DNF 窗口句柄，再把这个窗口放到固定位置，然后按这个窗口的客户区截图。**

3.1 当前主流程中的窗口处理

对应 `main.py` 的关键逻辑：

- `_find_dnf_window()` 会遍历所有可见窗口。
- 它根据标题和类名里是否包含 DNF 关键词来过滤候选窗口。
- 如果找到多个候选窗口，它会选面积最大的那个。
- `_place_window()` 会把窗口恢复显示，并移动到屏幕左上附近，方便后续稳定截图。
- `_get_client_region()` 会拿到客户区左上角坐标与宽高。
- `_capture_client()` 直接用 `pyautogui.screenshot(region=...)` 截这个区域。

也就是说，“绑定”在当前代码里，本质上不是“把输入系统绑进游戏进程”，而是“固定住我应该盯着哪一个窗口的哪一块区域”。

3.2 为什么这样也能工作

因为后面的识别与控制，都不要求程序真正进入游戏内部。只要它能稳定拿到画面，并且发送方向键和攻击键，它就能做出一个“像人在看屏幕玩游戏”的自动化行为。

3.3 代码里预留但没有接上的真正绑定能力

`dnf/olaplug/OLAPlugServer.py` 里有 `BindWindow()`、`BindWindowEx()`、`UnBindWindow()` 等接口，底层由 `OLAPlug_x64.dll` 提供能力。这类接口通常意味着更底层的窗口绑定、后台输入、截图或消息注入能力。

但是在当前源码搜索结果里，主流程并没有实例化 `OLAPlugServer`，也没有调用 `BindWindow()`。所以如果你是为了理解“现在这份程序怎么跑起来”，就不要把注意力主要放在 DLL 上。DLL 更像是一个还没正式接入、或者之前版本用过的能力库。

4. 程序是如何检测游戏画面的

这套程序不是只用一种检测方式，而是用了两套眼睛：

- 1. 大画面用 YOLO 目标检测。
- 2. 小地图用模板匹配。

4.1 大画面检测：YOLO 识别人物、怪物、门、物品

`detector.py` 负责加载模型 `shzn.pt`，然后对截图进行识别。它的作用不是“理解整个场景”，而是给出一堆框。每个框包含三部分：

- 类别，比如 `player`、`monster`、`boss`、`door`、`goods`、`money`
- 位置框 `xywh`
- 置信度 `conf`

4.2 YOLO 的核心参数是什么意思

参数	代码值	作用
<code>img_size</code>	640	送进模型的图片尺寸，越大通常越准，但越慢
<code>conf_thres</code>	0.75	置信度阈值，低于这个值的框不要
<code>iou_thres</code>	0.45	非极大值抑制阈值，用来去掉重复框

你可以把它理解成：YOLO 像一个“眼神很快的保安”，它看完一张图后告诉你：“玩家在这里，怪在这里，门在这里，掉落物在这里。”

4.3 小地图检测：模板匹配找房间

YOLO 负责看大画面，但它不负责判断“接下来应该去哪个房间”。这个问题交给 `minimap_nav.py`。

它的做法是：

- 1. 先把整张游戏截图裁成小地图区域。
- 2. 把小地图转成灰度图。
- 3. 拿预先准备好的图标模板去做 `cv2.matchTemplate`。
- 4. 找到英雄图标、Boss 图标、问号房、精英房、下层入口。

5. 再把图标所在的小地图像素位置，换算成“第几行第几列的房间”。

4.4 什么叫模板匹配

模板匹配很好理解：程序拿一张小图标，在小地图上滑来滑去，看哪里最像它。代码里用的是 `cv2.TM_CCOEFF_NORMED`，返回一个相似度热力图。相似度超过阈值，比如 `0.7`，就认为找到了。

大图：小地图截图

小图：Boss 图标模板

程序会问：

“把这个 Boss 小图放到小地图的每一个位置时，哪个位置最像？”

最像的位置，就是 Boss 房大概所在的位置。

4.5 房间坐标是怎么算出来的

程序把小地图看成一个网格。例如 `generic` 地图规格是 5 行 9 列。如果一个图标落在小地图某个像素位置，就用：

- 列号 = `x / cell_w`
- 行号 = `y / cell_h`

最终得到的是一个房间坐标，比如 `(2, 5)`，意思是第 3 行第 6 列。

5. 程序如何打怪、拾取物品、控制角色移动

5.1 游戏控制层的思想

`game.py` 里的 `Game.run()` 是整个动作系统的核心。你可以把它理解成一个“优先级决策器”。它每次只选一件最重要的事情做。

5.2 决策优先级

1. 先确认玩家位置
2. 如果正在移动但一直没动，执行卡住恢复
3. 如果有 Boss，打 Boss
4. 否则如果有普通怪，打怪
5. 否则如果有掉落物，去捡
6. 否则如果有金币，去捡
7. 否则如果有门，就按路线提示去找门
8. 如果什么都没有，就按后备策略移动或原地等待

5.3 程序如何知道该往哪个方向走

对于怪物、物品、门，它都先拿到目标中心点，再和玩家中心点做差：

- $dx = \text{目标}x - \text{玩家}x$
- $dy = \text{目标}y - \text{玩家}y$

然后根据 `dx`、`dy` 判断方向：

- 右边就是 `RIGHT`
- 左边就是 `LEFT`
- 上方就是 `UP`
- 下方就是 `DOWN`
- 斜着就在两个方向之间组合成 `RIGHT_UP`、`LEFT_DOWN` 之类

为什么需要斜方向

因为画面里的目标不一定和角色严格上下左右对齐，斜走能更快接近目标。

它不是物理模拟

这套代码没有复杂速度模型，基本上是“看目标在什么方向，就按什么方向键”。

5.4 打怪逻辑是怎么实现的

程序不会做很复杂的技能连招判断。它的思路非常直接：

- 1. 先找最近的 Boss，如果没有，再找最近的普通怪。
- 2. 如果怪物进入攻击范围，就先朝怪物方向转身，再按 `x` 攻击。
- 3. 如果还不在攻击范围内，就按目标方向慢慢靠近。

5.5 攻击范围和拾取范围

变量	值	含义
<code>_attack_x</code> / <code>_attack_y</code>	100 / 100	进入这个范围就开始攻击
<code>_move_x</code> / <code>_move_y</code>	20 / 20	进入这个范围就按 <code>f</code> 捡东西

5.6 角色移动是怎么发出去的

真正发键盘的是 `pydirectinput`。比如：

- `keyDown("right")` 表示按下右键
- `keyUp("right")` 表示松开右键
- `keyDown("x")` + `keyUp("x")` 表示攻击
- `keyDown("f")` + `keyUp("f")` 表示拾取

如果方向是 `RIGHT_UP`，代码会拆成两个键一起按，也就是同时按右和上。

5.7 为什么有一个 `_action_cache`

因为角色可能连续多帧都要往同一个方向走。如果每一帧都重新按一遍，会让动作很抖。所以程序记住“我上一帧正在按哪个方向”，如果这一帧方向没变，就尽量延续动作；如果方向变了，再把旧方向松开。

6. 程序是如何寻路的

6.1 这套代码里有两层“路”

这是理解源码最重要的地方之一。它不是从玩家当前位置一直规划到怪物旁边的一整条像素路径，而是分成两层：

1. **房间级路径**：当前房间应该去哪个相邻房间。这个由 A 星决定。
2. **画面级移动**：既然下一间房在右边，那当前房间里应该朝哪个门走。这个由门筛选和方向移动来完成。

所以，A 星在这套代码里解决的是“下一间房往哪边”，不是“房间内部怎么绕障碍物走”。

6.2 小地图上的地图模型

`minimap_nav.py` 里定义了 `MAP_SPECS`。每张地图都有：

- 小地图裁剪位置
- 小地图宽高
- 房间网格有几行几列
- `room_grid`，也就是一个二维数组

在这份代码里，`room_grid` 是全 0。0 代表可通行，1 一般会代表障碍。这说明当前实现里，A 星假定整张房间网格都是可以走的，没有给不同房间设置阻塞。

6.3 A 星做了什么

`minimap_astar.py` 的流程是一个标准的 A 星：

1. 把起点丢进开放列表 `open_list`。
2. 每次从里面拿出 `f` 值最小的节点。
3. 如果这个节点已经是终点，就回溯出路径。
4. 否则扩展它的邻居节点。
5. 计算邻居的 `g`、`h`、`f`。
6. 继续循环，直到找到终点或搜索失败。

6.4 A 星里的三个分数

名字	含义	在这份代码里的意思
<code>g</code>	起点到当前点已经走了多远	每走一步加 1
<code>h</code>	当前点到终点还估计有多远	用曼哈顿距离估算
<code>f</code>	总评分	<code>f = g + h</code>

6.5 曼哈顿距离是什么

曼哈顿距离公式是：

```
h(a, b) = |a.row - b.row| + |a.col - b.col|
```

因为这个网格里只允许上下左右走，不允许斜走，所以曼哈顿距离非常合适。你可以把它理解成“如果我只能走格子，至少还要走多少步”。

6.6 邻居扩展顺序为什么重要

代码里有一个很有意思的小技巧：邻居不是随便扩展的，而是和 `priority` 有关。比如当优先级是 `right` 时，邻居探索顺序就是：

```
右 -> 下 -> 上 -> 左
```

这不是改变 A 星的正确性，而是在多条路径同样短的时候，决定“更偏向先往哪边走”。所以它更像一种“同分时的偏好设置”。

7. A 星算法是如何和整个程序关联起来的

这是整份源码里最值得你抓住的一条线。很多初学者会以为“程序用了 A 星，所以角色的一举一动都由 A 星控制”。实际不是。

7.1 A 星接入点在哪里

接入点在 `minimap_nav.py` 的 `build_route_snapshot()`：

1. 先识别当前房间 `current_room`
2. 再确定目标房间 `target_room`
3. 调用 `next_direction(grid, current_room, target_room)`
4. 得到一个结果：`up` / `down` / `left` / `right`

关键点： A 星的输出不是“整条完整路径坐标”，而是“路径的第一步方向”。

7.2 为什么只取第一步

因为角色当前所在的是“大画面里的一个房间”，而不是整个副本网格上的抽象点。A 星知道“下一间房应该在右边”，但它不知道当前房间里门在屏幕上的哪个像素位置。所以它只能先告诉你一个房间级方向。

7.3 A 星之后发生了什么

小地图识别出：

当前房间 = (2, 3)

目标房间 = (2, 6)

A 星算出路径：

[(2,3), (2,4), (2,5), (2,6)]

程序不直接拿整条路径去控角色，
而是只看前两项：

(2,3) -> (2,4)

于是得到：

下一步方向 = right

然后这个 `right` 会继续传给门策略。门策略会问：“既然下一间房在右边，那当前画面里哪一扇门最像右边的门？”

7.4 门策略如何把 A 星方向转成可执行动作

`door_strategy.py` 做的事情是：

1. 拿到所有检测到的门。
2. 拿到玩家中心点。
3. 拿到 A 星给出的期望方向，比如 `right`。
4. 对每扇门计算 `dx`、`dy`。
5. 只把“真的在玩家右边”的门当作优先候选。
6. 同时按照“前进量越大越好、偏移越小越好、总距离越小越好”的顺序打分。
7. 选出分数最好的那扇门。

7.5 再往后怎么落到按键上

选出门之后，`Game.run()` 会继续做最后一步：它不关心 A 星了，它只关心“目标门中心点和玩家中心点之间差多少”。算出方向后，就调用 `_move()` 去按对应方向键。

7.6 整体串联图

截图

↓

YOLO 识别大画面

↓

模板匹配识别小地图

↓

当前房间 / 目标房间

↓

A 星求最短房间路径

↓

只取路径第一步方向

↓

用这个方向去筛当前房间的门

↓

得到门的屏幕坐标

↓

角色朝门移动

↓

进入下一间房

↓

重新截图，重新判断

这就是 A 星真正的关联方式：它不是直接控制角色，而是给“门选择模块”和“后备移动模块”提供方向提示。

7.7 一个你必须知道的现实细节

代码里虽然计算了 `selected_door_center`，但主流程传给 `Game` 的其实只有 `route_direction`，没有把这个门坐标真正传进去。也就是说，当前实现中：

- `MiniMapNavigator` 会做一次“选门”
- 但 `Game` 里又会根据方向提示自己再找一次门

这说明 A 星和门策略已经关联上了，但这个关联还不是最紧的版本。更严谨的做法应该是：导航器选好的具体门，直接交给控制层执行。

8. 所谓“容差算法”在这套代码里到底是什么

这份源码里没有一个单独叫“容差算法”的模块。它的“容差”其实分散在很多阈值和启发式规则里。你可以把它理解成：**为了让程序别那么死板，作者给很多判断都留了一点宽松空间。**

8.1 检测容差

位置	值	作用
YOLO 置信度阈值	0.75	防止识别出太多假目标
YOLO IOU 阈值	0.45	减少重复框
小地图模板阈值	0.7，部分图标 0.62	允许图标有轻微颜色和缩放变化

8.2 战斗和拾取容差

- 攻击不是要求角色和怪物中心完全重合，只要 X、Y 方向都进入 100 像素左右范围就开始打。
- 拾取不是完全踩到物品上，只要进入 20 像素范围就按 `f`。
- 方向判断里也允许一定的上下误差，比如 `abs(dy) < 20` 时仍然把目标当作左右方向。

8.3 门选择容差

在 `door_strategy.py` 里，参数 `margin=16.0` 就是一种容差。它表示：

- 如果期望往右走，那门的中心点要明显比玩家更靠右，至少超过 16 像素，才算真正匹配“右门”。
- 这样能减少把正上方、正下方或离得太近的门误判成右门。

8.4 卡住判断容差

程序并不会因为一两帧没动就认定卡住。它会保存最近几帧玩家位置，然后看位置散布范围：

- 如果最近 6 到 8 帧里，横向扩散不到 18 像素，纵向扩散也不到 18 像素
- 而程序又明明在尝试移动
- 那它才会把 `_stuck_count` 加 1
- 连续两次都这样，才真正触发卡住恢复

本质上：这不是一个数学上很复杂的容差算法，而是一组经验阈值系统。它们共同作用，让程序对误差、波动和偶发识别失误更宽容一点。

9. 重要算法逐个解释

9.1 最近目标算法

无论是怪物、金币、掉落物还是门，程序很多时候都会选“离玩家最近的那个”。距离公式就是欧几里得距离：

```
distance = sqrt(dx^2 + dy^2)
```

这不是最智能，但实现简单、速度快，也足够稳定。

9.2 方向分类算法

程序通过比较 `dx` 和 `dy`，把目标分成上、下、左、右、左上、右上、左下、右下。这是一个典型的规则判断算法，不是机器学习。

9.3 模板匹配算法

小地图识别用的是 OpenCV 的模板匹配。它把模板图标和小地图不同位置做相关性比较，找到最相似的位置。这适合图标固定、变化不大的 UI 元素。

9.4 A 星算法

A 星负责房间级的最短路搜索。由于网格简单、地图又小，它运行很快。而且因为启发函数是曼哈顿距离，所以在这种上下左右网格上很自然。

9.5 门打分算法

门选择不是只看最近，而是会计算一个三元评分：

```
score = (  
    -forward + reverse_penalty,  
    sideways,  
    abs_dx + abs_dy  
)
```

你可以这样理解：

- `forward` 越大越好，说明门更符合“前进方向”
- `sideways` 越小越好，说明横向偏差更小

- `abs_dx + abs_dy` 越小越好，说明总距离更近
- `reverse_penalty` 用来给反方向加惩罚，减少掉头

9.6 卡住恢复算法

这是一套小型状态机。如果判断卡住，它不会傻傻地一直按同一个方向，而是按模式轮换：

1. 先反方向退一下，再往下试
2. 再反方向退一下，再往上试
3. 再走一个小矩形：右、下、左、上
4. 最后再反方向和当前方向各试一下

这属于典型的“启发式脱困”，不是严格寻路，但在游戏自动化里很常见。

10. 你最该抓住的结论

10.1 这套程序真正的核心不是某一个函数，而是模块协作

你如果只盯着 A 星，会觉得看懂了又没完全看懂。你如果只盯着 YOLO，也会一样。真正要懂的是这条协作链：

窗口定位

- > 屏幕截图
- > YOLO 看大画面
- > 模板匹配看小地图
- > A 星给下一房间方向
- > 门策略找当前房间里最合适的门
- > 按键控制角色移动
- > 战斗 / 拾取 / 过门
- > 重新截图，进入下一轮

10.2 这份代码里的 A 星不是“全能大脑”

它只是房间级导航器。真正让角色动起来的是 `game.py` 里的方向判断和按键逻辑。真正让它知道往哪边过图的，是 `minimap_nav.py` 把小地图识别结果喂给了 A 星。真正让它在当前房间找到对应出口的，是 `door_strategy.py`。

10.3 这份代码的优点

- 模块分工清楚，适合学习。
- 没有把所有逻辑揉成一团，检测、导航、动作基本是分开的。
- A 星接得比较干净，思路容易扩展。
- 有卡住恢复、进门防回头、目标优先级这些实用小策略。

10.4 这份代码的局限

- 当前 A 星网格没有真实障碍信息，`room_grid` 基本全可通行。
- A 星只输出第一步方向，没有把整条路径状态持续管理起来。
- 导航器已经能选门，但控制层没有直接复用那个具体门坐标，关联还不够紧。
- 房间内部移动不是精细路径规划，更多是朝目标方向靠近。
- 容差主要靠经验阈值，换地图、换分辨率、换 UI 主题后可能需要重新调。

10.5 如果你继续学，推荐按这个顺序回看源码

1. 先看 `main.py`，只理解“每一帧发生什么”。
2. 再看 `detector.py`，理解识别结果长什么样。
3. 然后看 `minimap_nav.py`，理解小地图怎么变成房间坐标。
4. 接着看 `minimap_astar.py`，理解 A 星到底只产出了什么。
5. 最后看 `game.py`，理解动作是怎样按优先级执行的。

附录：源码阅读索引

主题	建议先看位置
主循环入口	<code>dnf/main.py</code> 中的 <code>main()</code>
找窗口 / 截图	<code>dnf/main.py</code> 中的 <code>_find_dnf_window()</code> 、 <code>_capture_client()</code>
YOLO 识别	<code>dnf/detector.py</code> 中的 <code>Detector.detect()</code>
小地图识别	<code>dnf/minimap_nav.py</code> 中的 <code>detect_room_markers()</code>
路线快照	<code>dnf/minimap_nav.py</code> 中的 <code>build_route_snapshot()</code>
A 星	<code>dnf/minimap_astar.py</code> 中的 <code>a_star()</code> 、 <code>next_direction()</code>
门评分	<code>dnf/door_strategy.py</code> 中的 <code>choose_best_door()</code>
动作总决策	<code>dnf/game.py</code> 中的 <code>Game.run()</code>
战斗 / 拾取	<code>dnf/game.py</code> 中的 <code>_kill_monster()</code> 、 <code>_pick_up()</code>
卡住恢复	<code>dnf/game.py</code> 中的 <code>_update_motion_state()</code> 、 <code>_recover_from_stuck()</code>
备用 DLL 绑定接口	<code>dnf/olaplug/OLAPlugServer.py</code> 中的 <code>BindWindow()</code>

如果你把这份书看完，只要再对照源码逐个函数去读，你就已经不是“完全看不懂代码”了。你会开始知道：每个文件是干嘛的，数据从哪来，到哪去，A 星到底在哪一层起作用。这就是读懂这类程序最关键的一步。

文档生成说明：本说明书根据当前工作区 `D:\yolo\Yolov11\dnf` 目录源码整理而成，重点反映“当前实际接线方式”，并区分了“已在主流程启用的逻辑”和“代码中存在但当前未接入的能力”。